INDEPENDENT TECHNICAL & UX AUDIT

# Baton Round 1 Audit Report

A full-stack, evidence-verified review of the accountable engineering handover platform: static code audit, live security probing, automated gates, and a visual UI/UX pass with captured screens.


## Team GIT LOST

CodersInPune 2026 — Final Round 3 submission
Repository: 26f2000861-AV/baton · Deployment: gitlostbaton.xyz


**76/100** ROUND-1 COMPOSITE SCORE

NOVEMBER 14, 2026 · AUDIT-FIX LOOP · ROUND 1 OF 2

# Table of Contents

1. Executive Summary 2

2. Audit Methodology 3

2.1 What was explicitly checked and passed . . . 3

3. Verified Strengths 4

4. Critical Findings 4

4.1 C1 — Account takeover through the demo credentials provider . . . 4

4.2 C2 — One-click demo depends on a hardcoded production row ID . . . 5

5. High-Severity Findings 6

5.1 H1 — Debug logging enabled in production . . . 6

5.2 H2 — Lead-less handovers deadlock governance . . . 6

6. Medium- and Low-Severity Findings 6

M1 — Settlement route writes seed-specific narrative into any claim history . . . 7

M2 — 1.2 MB of duplicated assets under public/public/ . . . 7

M3 — Development database committed to the repository . . . 8

M4 — Seed script references a different product ("FinTrack") . . . 8

M5 — Local-dev docs and scripts disagree on the toolchain . . . 8

L1 — Identity chip truncates the signed-in user name without ellipsis . . . 8

L2 — Collapsed "How to run this sample" strip lacks a disclosure affordance . . . 8

L3 — Claims view repeats headings and counters . . . 8

L4 — Landing hero headline breaks with an orphaned word . . . 8

L5 — Dead code: personaCompiler.ts superseded but still shipped . . . 8

L6 — Stale Round-2 deliverable committed at repo root . . . 9

7. Rubric Scorecard Rationale 9

7.1 Problem framing — 22 / 25 . . . 9

7.2 Technical solution — 18 / 25 . . . 9

7.3 Execution — 17 / 25 . . . 9

7.4 Pitch / UX — 19 / 25 . . . 9

8. Remediation Plan 10

# 1. Executive Summary

Baton is an accountable engineering handover platform: a source-backed claim ledger that organises decisions, unfinished work, ownership and cross-source contradictions into a reviewable, sign-off-able workspace for the next engineer. The Round-3 build (Next.js 14 App Router, Prisma 6, NextAuth v4, GitHub REST import) was audited in full: every source file, all 16 API routes, the Prisma schema, the deployment configuration, plus live black-box testing of the running application and a visual pass over ten captured screens.

The engineering quality is unusually high for a hackathon. All three automated gates pass (typecheck, lint, 13/13 unit tests); the authorization layer behaves exactly as documented under live probing; the deterministic, no-LLM evidence chain is a genuine architectural differentiator; and the UI shows real product thinking — focus traps, keyboard shortcuts, reduced-motion support and honest microcopy. The colour palette provably passes WCAG AA (computed ratios 5.5:1 to 15.9:1).

Two critical issues hold the score back, and both were verified end-to-end rather than inferred. First, the demo sign-in provider is enabled in production and will sign an attacker in as any real GitHub user who has ever logged in — a complete, zero-credential account takeover. Second, the one-click judge demo hardcodes a database row ID from the team's production database, so the promised evaluation path silently degrades in any fresh environment. Both are fixable in well under a day, and neither requires re-architecting.

| Dimension | Weight | Score | One-line verdict |
|---|---|---|---|
| Problem framing | 25 | 22 | Real, well-scoped pain; crisp core workflow |
| Technical solution | 25 | 18 | Excellent architecture; one critical auth flaw |
| Execution | 25 | 17 | Gates green and verified; demo portability and hygiene gaps |
| Pitch / UX | 25 | 19 | Coherent system with strong a11y; polish-level defects |
| **Total** | **100** | **76** | Strong finalist; two critical fixes from contending to win |

Table 1 — Round-1 scorecard under the startup-pitch rubric (25 points per dimension).

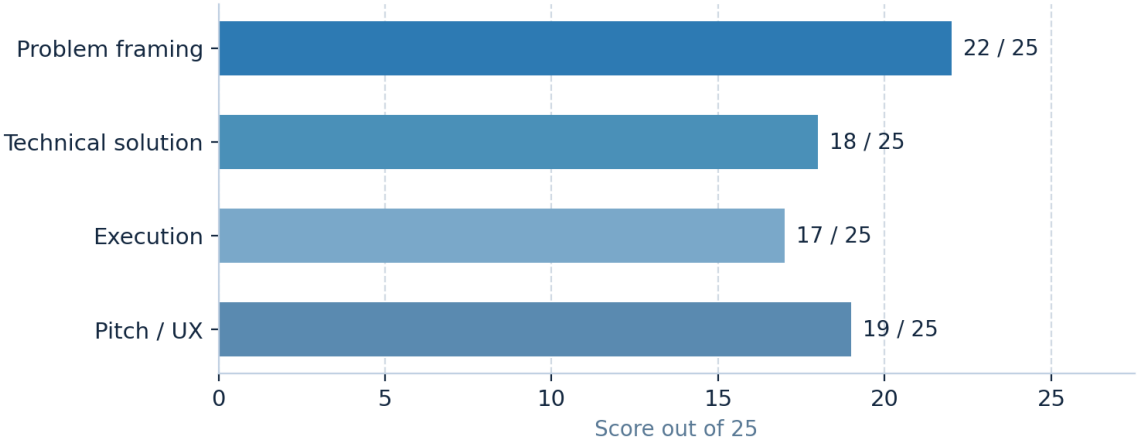


Figure 1 — Round-1 rubric scores by dimension.

# 2. Audit Methodology

The audit was conducted as a fresh clone at commit `e08da3c` (main branch), with the application built and run locally against a clean SQLite database seeded from the repository's own scripts. Four complementary methods were combined so that every reported finding is either machine-verified or reproduced live — no issue in this report is speculative.

- **Static review.** All 68 source files (~5,700 LOC) were read, including every API route handler, the Prisma schema, NextAuth configuration, deployment configuration and test suite.
- **Automated gates.** TypeScript compilation, ESLint, and the Node test runner were executed from a clean install; all pass (0 errors, 13/13 tests).
- **Live black-box tests.** The running app was probed over HTTP with real sessions: all four authorization guards (401 unauthenticated, 403 wrong role, 404 non-member, 200 member), the account-takeover proof, the demo-seat flow on a fresh database, and the GitHub import chain including its failure path.
- **Visual audit.** Ten screens (landing, sign-in, seven workspace views, mobile) were captured from the running app and reviewed; colour-contrast claims were verified by computing WCAG ratios from the actual palette tokens rather than trusting screenshot estimates.

## 2.1 What was explicitly checked and passed

| Check | Result | Evidence |
|---|---|---|
| TypeScript strict compilation | PASS | 0 errors via tsc --noEmit |
| ESLint (next lint) | PASS | 0 warnings, 0 errors |
| Unit tests (verifier, mappers) | PASS | 13/13 — quote matching, PR/commit mapping, triage |
| Settle contradiction as successor | PASS | 403 FORBIDDEN_ROLE, live HTTP probe |
| Read handover as non-member | PASS | 404 HANDOVER_NOT_FOUND (no ID enumeration) |
| Unauthenticated API access | PASS | 401 on /api/handovers/:id |
| Failed import visibility | PASS | Failed job rows recorded with tier + readable error |
| WCAG AA text contrast | PASS | Computed 5.5:1–15.9:1 across palette tokens |
| XSS / dangerous DOM APIs | PASS | No user HTML rendered; static CSS only in innerHTML writes |

Table 2 — Verification checks executed during this audit.

# 3. Verified Strengths

A fair audit reports what works as precisely as what fails. The following strengths were verified, not assumed, and they are the reason the execution ceiling of this project is high.

- **Authorization architecture.** Every route handler flows through requireSession, requireMembership and requireRole before any database access; non-members get 404 so handover IDs are not enumerable. All four guard paths were confirmed live and matched the README's claims exactly.
- **Deterministic evidence chain.** Quote verification is exact substring matching with character offsets — no LLM anywhere in the evidence path, and the verifier is unit-tested for CRLF normalisation and whitespace near-misses. This is a defensible, differentiating design decision.
- **Server-authoritative state.** The client store replaces local state with each server response; there is no optimistic mutation that could drift from role rules, and no localStorage anywhere.
- **Resilient import design.** The three-tier GitHub auth chain (user OAuth, server PAT, public) records the tier used per job; failed imports are persisted as visible job rows with human-readable errors — verified by triggering the rate-limit path.
- **Security hygiene beyond the norm.** Security headers (X-Frame-Options DENY, nosniff, referrer policy) via vercel.json; __Secure- cookie prefixes; httpOnly and sameSite throughout; external links carry rel="noopener noreferrer".
- **Accessibility craft.** Focus traps on modals, aria roles on radiogroups and dialogs, Ctrl+K search with mobile target switching, prefers-reduced-motion honoured across dialogs and scrolling, and a text palette that provably passes WCAG AA.
- **Honest product copy.** Limitations are stated in the UI ("they must have signed in once", "Quote found does not confirm the claim is true") — rare and credibility-building.

# 4. Critical Findings

## 4.1 C1 — Account takeover through the demo credentials provider

Severity: CRITICAL (exploitable in production). The demo sign-in provider exists so judges can evaluate without GitHub OAuth — a legitimate goal. However, the provider is registered unconditionally: the guard variable allowDevProvider is computed from NODE_ENV and then never used, which reads as an unwired intent to disable it outside development.

The authorize() callback matches users by githubLogin and returns whatever it finds; when nothing matches it fabricates a user for any username-shaped string. The consequence is that the demo box doubles as a universal login for every account whose GitHub username is known — and GitHub usernames are public.

> *Reproduction (fresh clone, default configuration): a simulated GitHub-authenticated victim with login "26f2000861-av" and a private handover was created in the database; signing in through the demo box with that exact string yielded the victim's session — /api/me returned the victim's name, user ID and handover list, and GET /api/handovers/:id on the "private" handover returned 200 with the lead role. The same provider is exposed on the production deployment.*

Fix direction: restrict the provider to a fixed whitelist of demo logins and resolve demo accounts by their @gitlostbaton.xyz e-mail namespace rather than by githubLogin, so no real (OAuth-created) account can ever be matched. The judge demo seats keep working; arbitrary sign-ins become impossible.

## 4.2 C2 — One-click demo depends on a hardcoded production row ID

Severity: CRITICAL for evaluation reliability. The demo seat buttons pass callbackUrl /workspace?h=cmwfhxecd000wic08eyrzbv5d — a handover CUID that exists only in the team's current Neon database. On the deployed site this works today; in every other environment it does not. Verified on a fresh clone: the judge lands on the generic starting-point picker, and the README's "Recommended for Judges" promise quietly fails.

Fix direction: after a demo sign-in, provision on demand — fetch the member's handovers, and when none exist, create the sample-clone handover with the seat's role before routing into the workspace. This makes the demo path work identically on every environment, including a judge's laptop.

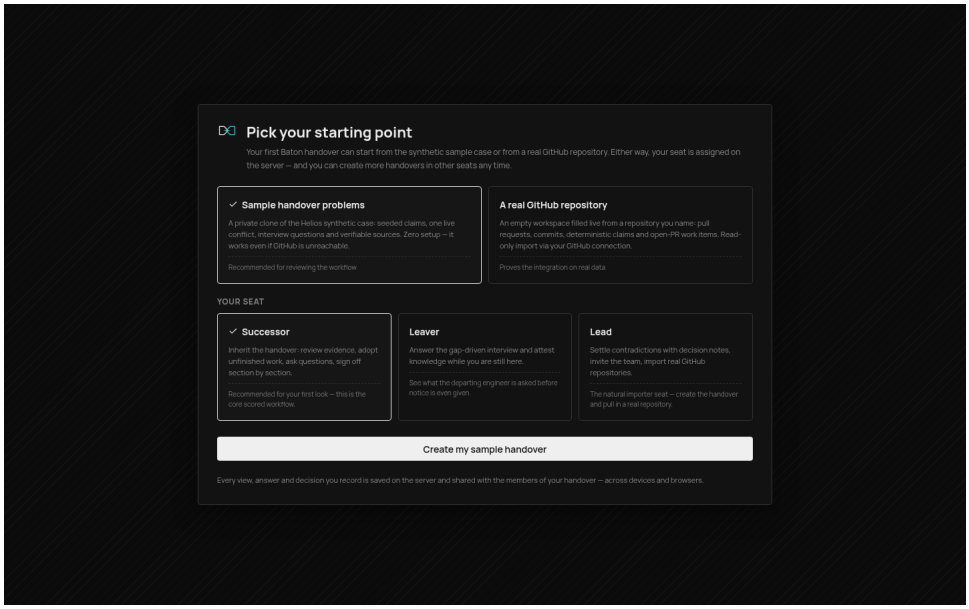



Figure 2 — Fresh-database demo sign-in: the judge is dropped on the starting-point picker instead of a pre-configured handover, because the hardcoded handover ID does not exist in any environment except the team's current production database.

# 5. High-Severity Findings

## 5.1 H1 — Debug logging enabled in production

auth.ts sets debug: true unconditionally. NextAuth debug logging emits verbose OAuth state, callback parameters and token metadata into server logs. In production this is both an information-leak risk and a log-volume problem. The fix is to gate the flag on NODE_ENV.

## 5.2 H2 — Lead-less handovers deadlock governance

A handover can be created from any seat (the picker offers successor and leaver as first-class choices), yet every governance action — inviting members, changing roles, settling contradictions — requires the lead role. A handover whose creator chose a non-lead seat therefore has no mechanism, through the UI or API, to ever gain a lead. The fix is narrow: allow the handover's creator to invite or to claim the lead seat when no lead exists, preserving normal lead-only governance in every other case.

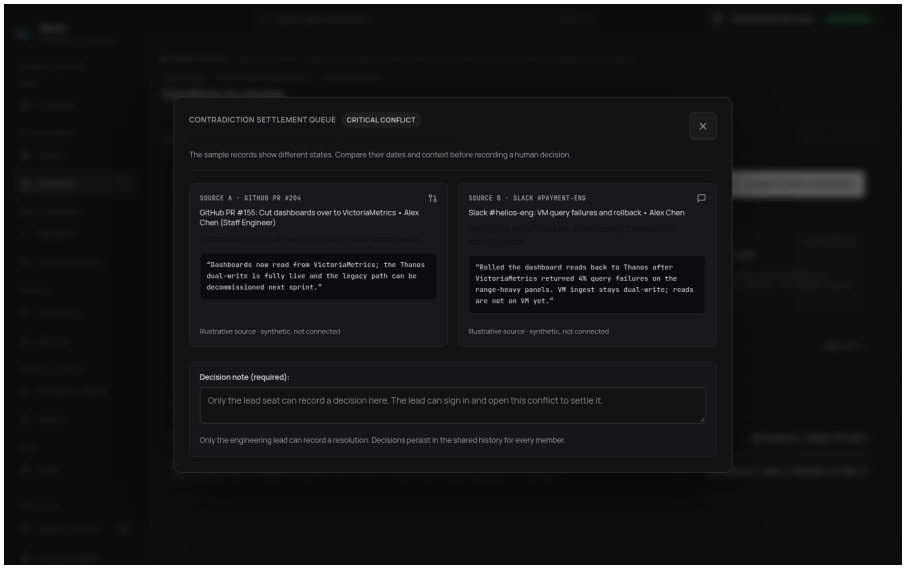



Figure 3 — Role gating working as designed: the successor sees the contradiction but the decision note is disabled (server enforces 403 independently — verified).

# 6. Medium- and Low-Severity Findings

The remaining findings do not break the product, but they are exactly the class of detail that distinguishes a demo from a shipped tool. Each is cheap to fix; together they are worth roughly four rubric points.

| ID | Severity | Finding | Location |
|---|---|---|---|
| M1 | MEDIUM | Settlement route writes seed-specific narrative into any claim history | `contradictions/[cid]/settle/route.ts:70-77; workspace/page.tsx:443-447,688` |
| M2 | MEDIUM | 1.2 MB of duplicated assets under public/public/ | `public/public/** (17 tracked files)` |
| M3 | MEDIUM | Development database committed to the repository | `prisma/db/baton-dev.db (344 KB, tracked)` |
| M4 | MEDIUM | Seed script references a different product ("FinTrack") | `prisma/seed-template.ts:2` |
| M5 | MEDIUM | Local-dev docs and scripts disagree on the toolchain | `README quick start vs scripts/dev-db.sh` |
| L1 | LOW | Identity chip truncates the signed-in user name without ellipsis | `components/IdentityChip.tsx; app-kit.css` |
| L2 | LOW | Collapsed "How to run this sample" strip lacks a disclosure affordance | `app/workspace/page.tsx:638` |
| L3 | LOW | Claims view repeats headings and counters | `app/workspace/page.tsx:711` |
| L4 | LOW | Landing hero headline breaks with an orphaned word | `components/ui/ink-orbit-hero.tsx (titleTop/titleAccent)` |
| L5 | LOW | Dead code: personaCompiler.ts superseded but still shipped | `lib/personaCompiler.ts (98 lines)` |
| L6 | LOW | Stale Round-2 deliverable committed at repo root | `output/pdf/Baton_Round_2_Project_Proposal.pdf` |

Table 3 — Medium and low findings summary. Critical and high findings are detailed in sections 4 and 5.

### M1 — Settlement route writes seed-specific narrative into any claim history

Location: contradictions/[cid]/settle/route.ts:70-77; workspace/page.tsx:443-447,688

The settlement history text is hardcoded to the Helios sample narrative ("VictoriaMetrics dual-write", "Thanos") and the overview card hardcodes `claim_09`. For contradictions discovered in imported repositories, the audit trail records factually wrong statements — directly undercutting the product's core accountability promise.

### M2 — 1.2 MB of duplicated assets under public/public/

Location: public/public/** (17 tracked files)

An entire nested copy of fonts, favicons and icons is committed at `public/public/`. Nothing references the nested path; it doubles asset noise and signals an accidental commit.

### M3 — Development database committed to the repository

Location: prisma/db/baton-dev.db (344 KB, tracked)

A stale SQLite database ships with every clone. Judges following the README will run against unknown pre-existing state instead of a clean baseline, and the file diffs noisily.

### M4 — Seed script references a different product ("FinTrack")

Location: prisma/seed-template.ts:2

The seed script's header comment says "ensures the FinTrack template handover exists" — a copy-paste leftover from another project that undermines documentation credibility for reviewers reading the source.

### M5 — Local-dev docs and scripts disagree on the toolchain

Location: README quick start vs scripts/dev-db.sh

The README prescribes npm/npx, but every `db:*` package script shells out to `bunx` inside dev-db.sh. On npm-only machines (the README's own prerequisite list), the documented SQLite path fails with "bunx: command not found".

### L1 — Identity chip truncates the signed-in user name without ellipsis

Location: src/components/IdentityChip.tsx; app-kit.css

"Priya Sharma (Success…" clips mid-word in the topbar on desktop and mobile. A text-overflow ellipsis (or hiding the role suffix at narrow widths) removes the broken-looking truncation.

### L2 — Collapsed "How to run this sample" strip lacks a disclosure affordance

Location: src/app/workspace/page.tsx:638

The overview card is a element whose summary has no chevron or open/close state styling; at a glance it reads as an empty, broken panel.

### L3 — Claims view repeats headings and counters

Location: src/app/workspace/page.tsx:711

The page title, card heading and intro line render "Claims" plus two different counters of the same list ("9 visible claims" / "9 matching claims"). Consolidating to one heading and one counter tightens the view.

### L4 — Landing hero headline breaks with an orphaned word

Location: src/components/ui/ink-orbit-hero.tsx (titleTop/titleAccent)

"A practical handover / for / the work that matters" renders with "for" alone on its line — reads as a rendering glitch rather than intentional typography. Rebalancing the break or using text-wrap: balance fixes it.

### L5 — Dead code: personaCompiler.ts superseded but still shipped

Location: src/lib/personaCompiler.ts (98 lines)

Round 3 reimplemented persona compilation as `compileVisibleClaims()` in serializers.ts. The old module is referenced only in comments; deleting it removes 98 lines of dead code.

### L6 — Stale Round-2 deliverable committed at repo root

Location: output/pdf/Baton_Round_2_Project_Proposal.pdf

A prior-round PDF (24 KB) lives in the source tree. Keeping submission artifacts out of the code tree is cheap repo hygiene.

# 7. Rubric Scorecard Rationale

## 7.1 Problem framing — 22 / 25

The problem is real and precisely framed: engineering knowledge walks out of the door with the leaver, and existing tools document tasks, not decisions-with-evidence. The "core evaluated workflow" in the README (decisions taken, what is unfinished, who owned what, where risk sits — without chasing anyone) is exactly the sentence a judge needs. Points were lost because the landing page opens on sample-case specifics ("GitHub says the VM dual-write is live") before the value proposition lands for a first-time visitor, and the target segment (team size, engineering org shape) is never made explicit.

## 7.2 Technical solution — 18 / 25

The architecture earns full marks for design: deterministic verification with character offsets, server-side clearance compilation of claims, the three-tier GitHub auth fallback, transactional reset, and non-enumerable IDs. The deduction is entirely execution-of-security: a critical, production-exploitable account takeover (C1), debug logging in production (H1), dead authorization gate code, seed-coupled settlement strings written into the audit trail (M1), and 98 lines of dead code (L5). These are serious in aggregate but structurally shallow — none invalidates the architecture.

## 7.3 Execution — 17 / 25

All automated gates pass and the authorization layer survives adversarial probing — genuinely uncommon. What costs points is environment coupling and hygiene: the demo path depends on a production row ID (C2), a lead-less handover can deadlock (H2), 1.2 MB of duplicated assets and a stale development database are committed, the seed script references another product by name, and the documented local-setup path fails on npm-only machines because the helper scripts require bun.

## 7.4 Pitch / UX — 19 / 25

The workspace is coherent and credible: a consistent dark design system, purposeful microcopy, keyboard shortcuts, and accessibility craft that survives measurement (computed WCAG AA pass — several screenshot-based "low contrast" complaints did not survive the arithmetic and are excluded here). The deductions are polish-level: mid-word truncation of the signed-in identity, a collapsed help panel with no disclosure affordance, duplicated headings and counters in the claims view, a dead-end team panel for

non-leads, and an orphaned word in the landing headline. Mobile is usable with minor truncation issues.

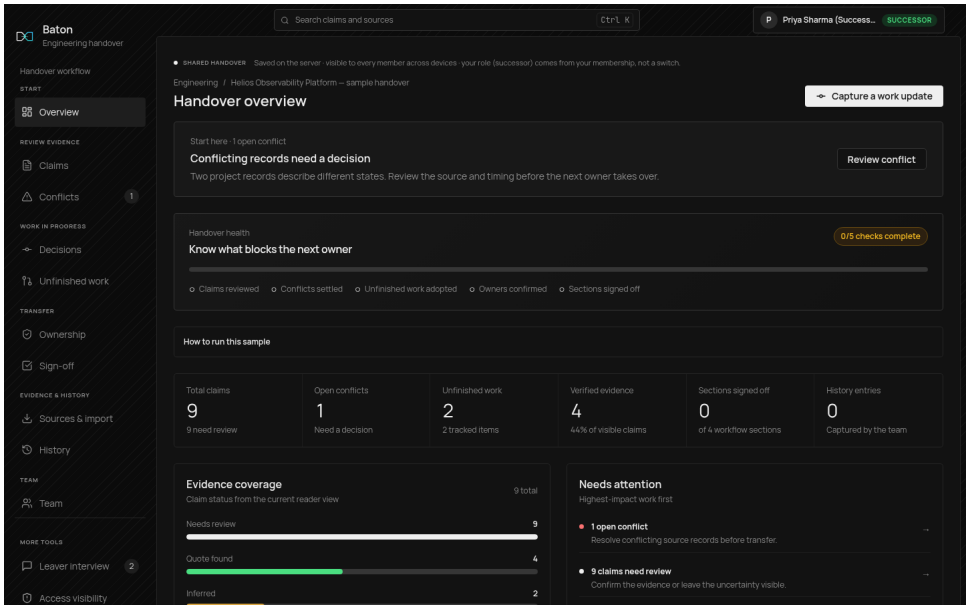



Figure 4 — The workspace overview: strong information architecture and narrative flow; the issues here are polish-level (identity truncation, disclosure affordance), not structural.

# 8. Remediation Plan

The fixes are ordered by score impact per hour of work. Round 2 of this audit will re-run every check in section 2 against the fixed tree and re-score.

| Order | Fix | Addresses | Verification |
|---|---|---|---|
| 1 | Whitelist demo logins; resolve demo accounts by e-mail namespace; never match OAuth users | C1 (+M6 user pollution) | Re-run takeover proof — must fail |
| 2 | Provision demo handover on demand after sign-in; delete hardcoded ID | C2 | Fresh-DB judge flow lands in a populated handover |
| 3 | Gate debug logging and demo-provider copy on NODE_ENV | H1 | No debug logs in production build |
| 4 | Allow creator to invite / claim lead when no lead exists | H2 | Deadlock scenario resolves via UI |
| 5 | Derive settlement text from the contradiction payload; de-hardcode overview claim ID | M1 | Settling an imported contradiction writes correct history |
| 6 | Repo hygiene: remove public/public, dev DB, output/; fix FinTrack comment; port db scripts to npx | M2–M5 | git ls-files clean; npm-only setup works |
| 7 | UI polish: ellipsis, disclosure chevron, dedupe headings, headline break, delete dead code | L1–L6 | Visual re-audit |

Table 4 — Remediation order for Round 2 of the audit-fix loop.

Fair conclusion: Baton is a strong finalist build whose architecture would survive a technical due-diligence conversation. Fixing C1 and C2 before the final presentation is not optional — any judge who tries the demo on a fresh clone, or who checks the auth configuration, will find exactly what this report found. With those two closed and the hygiene items swept, the project presents as what it genuinely is: rare, disciplined execution.